

Localo

Product, Roles, Features, Workflows and Roadmap Blueprint

Codex-ready product context for backend-first implementation

Owner	Businice Developers
Product	Localo
Primary goal	Local shop discovery and management platform
Implementation strategy	Backend first: NestJS + PostgreSQL + Prisma + REST API
Version	1.0 - 2026-06-03

How to use this PDF

Feed this PDF to Codex before backend implementation together with the DB Blueprint and NestJS Backend Blueprint. This file explains what Localo is, who uses it, what each role can do, and the workflow/roadmap order. It prevents Codex from guessing product behavior.

1. Document Map

- Product definition: what Localo is and what it is not.
- User roles: Admin, Shop Owner, Customer/User, Guest, Support/Operator, and System Jobs.
- Feature catalogue: role-wise features and backend responsibilities.
- Workflow catalogue: end-to-end flows from shop creation to user discovery.
- Roadmap: backend-first build order for Codex tasks.
- Codex guidance: how to feed product context without wasting usage.

Decision

This PDF is intentionally product-focused. Use the DB Blueprint for tables/columns/relations and the NestJS Backend Blueprint for APIs, DTOs, modules and coding rules.

2. What is Localo?

Localo is a local discovery and shop management platform. It helps customers discover nearby shops, browse products or services, check availability/business hours, view location details, and contact or visit the shop.

For shop owners, Localo provides a structured dashboard to manage shop profile, products, categories, pricing, discounts, media, business hours, location, and customer-facing information.

For platform admins, Localo provides control over shop creation, shop approvals, owner access, categories, support tickets, platform settings, commissions, analytics, and moderation.

The first implementation should be backend-first. We will build the database, auth, roles, APIs and core workflows before focusing deeply on UI polish.

Simple product sentence

Localo is a local marketplace/discovery system where Admins manage the platform, Shop Owners manage their shop data, and Customers discover shops and products near them.

2.1 Product Principles

- System-first: the backend data model and workflows should be clean before UI implementation.
- Role-first: every API must clearly know which role can access it.
- Admin-controlled onboarding: in MVP, Admin can create shops and invite or assign shop owners.
- Location-aware: shop discovery depends heavily on city, area, geolocation, and searchable tags/categories.
- Search-ready data: categories, tags, slugs, keywords and status fields must be designed for discovery.
- Long-term scalable: support commission, shop pause/suspend, analytics, notifications and payments later.

2.2 What Localo is not in MVP

- Not a full e-commerce checkout system in MVP.
- Not a delivery/logistics platform in MVP.
- Not a complex inventory/warehouse system in MVP.
- Not a marketplace with automatic seller registration in MVP; Admin-created shops come first.
- Not a heavy ERP system. Keep shop management simple, fast and mobile-friendly.

3. Product Scope and Core Objects

Object	Meaning	Owned/managed by	Why it matters
User	Any person authenticated in Localo.	System/Admin	Base identity for admins, owners and customers.
Role	Permission identity such as ADMIN, SHOP_OWNER, CUSTOMER.	System/Admin	Controls API access and screens.
Shop	Business listing visible to customers.	Admin + Shop Owner	Main asset of the platform.
Shop Owner	Business user responsible for shop data.	Admin	Manages products, hours, profile and support.
Location	Address, city, area, coordinates and service area.	Admin + Shop Owner	Drives discovery and maps.
Business Hours	Opening/closing schedule and special closures.	Shop Owner	Customers need to know when shops are open.
Category	Platform or shop-level grouping.	Admin + Shop Owner	Search, filtering and browsing.
Product/Service	Item, offering, menu/service/product listing.	Shop Owner	Main customer-facing content.
Media	Images/logos/banners/product photos.	Shop Owner/Admin	Improves trust and browse experience.
Support Ticket	Help request raised by owner or user.	User/Owner/Admin	Operational support workflow.
Commission/Subscription	Commercial relationship with shop.	Admin/System	Revenue and shop pause logic.

3.1 Core Product Statuses

Area	Statuses	Usage
Shop	DRAFT, PENDING_REVIEW, ACTIVE, PAUSED, SUSPENDED, REJECTED, ARCHIVED	Controls whether a shop is visible and manageable.
Product	DRAFT, ACTIVE, INACTIVE, OUT_OF_STOCK, ARCHIVED	Controls customer visibility and listing quality.
Owner invite	PENDING, ACCEPTED, EXPIRED, REVOKED	Used when Admin invites owner to manage shop.
Support ticket	OPEN, IN_PROGRESS, WAITING_FOR_USER, RESOLVED, CLOSED	Controls support queue.
Commission/payment	PENDING, PAID, OVERDUE, WAIVED, FAILED	Used to track commercial obligations.
Notification	QUEUED, SENT, FAILED, READ	For future messaging.

4. User Roles

Role	Who they are	Primary objective	Main screens
Super Admin	Platform owner or technical operator.	Full control over platform data, settings and admin users.	Admin dashboard, settings, audit logs, user management.
Admin	Operations team managing shops and support.	Create shops, manage owners, review content, handle support.	Dashboard, Shops, Users, Support, Categories, Commission.
Shop Owner	Business owner or manager.	Keep shop profile, hours, products and customer content updated.	Owner dashboard, Shop profile, Products, Hours, Support, Analytics.
Customer/User	End user browsing Localo.	Find shops/products nearby and interact with listings.	Home, Search, Shop details, Product details, Favorites, Profile.
Guest	Unauthenticated visitor.	Browse public content with limited actions.	Home, Search, Shop/Product detail with login prompts.
Support/Operator	Optional internal support role.	Resolve tickets without full platform admin power.	Support queue, ticket details, user/shop lookup.
System Jobs	Automated backend tasks.	Maintain computed states, reminders, expiry and analytics.	No UI; background jobs and scheduled tasks.

4.1 Role Access Matrix

Feature Area	Super Admin	Admin	Shop Owner	Customer	Guest
Admin dashboard	Full	Full/limited	No	No	No
Create shops	Yes	Yes	No in MVP	No	No
Manage assigned shop profile	All shops	All shops	Own shops only	No	No
Manage products	All shops	All shops	Own shops only	No	No
Browse shops/products	Yes	Yes	Yes	Yes	Limited
Favorites	No need	No need	No need	Yes	No
Reviews	Moderate	Moderate	Reply/report later	Create/edit own	Read only
Support tickets	All	All/assigned	Own shop tickets	Own tickets	No or contact form
Commission/payment status	All	All	Own shop only	No	No
Platform settings	Full	Limited	No	No	No

5. Role Feature Catalogue

5.1 Admin Features

Feature	Description	Backend requirement
Admin auth	Login securely with role-based dashboard access.	JWT auth, refresh tokens, role guards, audit login events.
Dashboard	View total shops, active shops, pending issues, support queue, revenue/commission summary.	Aggregated statistics endpoints with filters.
Shop management	Create, edit, approve, pause, suspend and archive shops.	Shop CRUD, status transition rules, audit logs.
Owner management	Invite or assign shop owners to shops.	Owner invites, user creation, ownership relations.
Category management	Create platform categories, tags and featured categories.	Category CRUD, slug uniqueness, ordering.
Product moderation	View or manage products for quality control.	Admin product access, status updates, content flags.
Support management	Handle tickets from shop owners and customers.	Ticket CRUD, comments, attachments, status transitions.
Commission management	Track shop commissions/subscriptions and pause unpaid shops if needed.	Commission records, payment status, overdue logic.
Settings	Configure platform-wide rules, fees, visibility and feature flags.	Settings table, typed keys, guarded updates.
Audit logs	Track important admin and owner actions.	AuditLog model with actor, action, entity, metadata.

5.2 Shop Owner Features

Feature	Description	Backend requirement
Owner login	Login after admin invitation or account creation.	Auth, invite acceptance, owner role assignment.
Owner dashboard	See shop status, profile completion, product count, support, views/analytics later.	Owner-scoped aggregate endpoints.
Shop profile	Manage logo, banner, description, phone, WhatsApp, email, social links.	Shop profile DTOs, media relation, validation.
Location	Set address, city, area, pincode, coordinates and map location.	Shop location table and geospatial-friendly indexes.
Business hours	Set opening/closing hours, weekly schedule and special closures.	BusinessHours and SpecialHours endpoints.
Product catalogue	Create products/services with price, discount, category, tags, images and status.	Product CRUD, media, category relation, price validation.
Availability	Mark products active/inactive/out of stock and shop open/closed.	Status fields and transition validation.
Support tickets	Raise support tickets to admin.	Ticket creation scoped to owner/shop.
Notifications	Receive alerts about shop approval, unpaid commission, support replies and moderation.	Notification entity and delivery-ready design.
Commission view	See current commission/subscription status and unpaid dues.	Owner-scoped commission read endpoints.

5.3 Customer/User Features

Feature	Description	Backend requirement
Browse home	See categories, nearby shops, featured shops and recommended products.	Public discovery APIs with filters.
Search	Search shops/products by keyword, category, tag, area and location.	Search endpoints, indexes, normalized fields.
Location preference	Use current location or manually selected city/area.	User location preference model; location filters.
Shop details	View shop profile, address, hours, products, media, contact buttons and reviews.	Public shop details endpoint with computed open status.
Product details	View product/service details, price,	Public product details endpoint.

	discount, photos, category and shop information.	
Favorites	Save favorite shops/products.	FavoriteShop/FavoriteProduct tables.
Reviews/ratings	Rate/review shops/products if enabled.	Review model, moderation status, ownership validation.
Support	Raise customer support ticket or report issue.	Ticket creation scoped to customer.
Notifications	Receive updates for saved shops, support replies and platform messages later.	Notification preference model later.

5.4 Guest Features

- Can browse public shops and categories if the product allows guest discovery.
- Can view shop and product detail pages with limited personalization.
- Cannot save favorites, write reviews, raise authenticated tickets or manage shop data.
- Should see login/signup prompts when attempting customer-only actions.

5.5 Support/Operator Features

- Can view assigned support tickets and update ticket status.
- Can add internal notes and replies depending on permission.
- Can lookup basic shop/user context for support resolution.
- Cannot change platform settings, commission rules or delete core data unless promoted to Admin.

6. Workflow Catalogue

Important implementation rule

Every workflow below should map to API endpoints and database state transitions. Codex should not invent new workflows unless explicitly requested.

6.1 Admin creates a shop

1. Admin logs in and opens Shops section.
2. Admin creates a new Shop with name, category, city/area, contact details and initial status DRAFT or PENDING_REVIEW.
3. Admin optionally creates/invites a Shop Owner and links owner to the shop.
4. System stores audit log for shop creation and owner assignment.
5. Shop is not visible to customers until status becomes ACTIVE.

Backend mapping

Required backend: POST /admin/shops, POST /admin/shop-owner-invites, shop status rules, ownership relation, audit logs.

6.2 Shop owner accepts invite and completes setup

6. Owner receives invite link or credentials from Admin.
7. Owner accepts invite, creates password or logs in.
8. Owner lands on setup checklist: profile, location, hours, products, media.
9. Owner updates missing shop details.
10. System computes profile completion percentage.

Backend mapping

Required backend: invite acceptance, owner auth, shop profile endpoints, completion calculation.

6.3 Shop goes live

11. Admin reviews shop data quality.
12. Admin changes shop status to ACTIVE.
13. System records status transition and audit log.
14. Shop becomes visible in public search and discovery APIs.
15. Owner receives notification that shop is live.

Backend mapping

Required backend: PATCH /admin/shops/:id/status, public visibility filters, notifications.

6.4 Shop owner manages products

16. Owner opens Products section.
17. Owner creates product/service with name, category, price, discount, description, tags and images.
18. Owner chooses DRAFT or ACTIVE status.
19. System validates price, discount and ownership.
20. Active products appear under shop detail and public product search.

Backend mapping

Required backend: owner-scoped product CRUD, product media, category/tag handling, public filters.

6.5 Customer discovers a shop/product

21. Customer opens app and selects/current location is resolved.

22. Customer browses categories or searches keyword.
23. System returns active shops/products based on location, category, tags, and search ranking.
24. Customer opens shop or product detail.
25. Customer contacts shop, navigates to location, saves favorite or reviews if logged in.

Backend mapping

Required backend: public search API, geolocation filters, open status computation, favorites/reviews.

6.6 Support ticket lifecycle

26. Shop Owner or Customer creates support ticket with type, priority and message.
27. Admin or Support Operator sees ticket in queue.
28. Admin replies, changes status, requests more information or resolves.
29. Requester receives notification when reply/status changes.
30. Ticket is closed after resolution.

Backend mapping

Required backend: tickets, ticket messages, attachments later, status transitions, notification records.

6.7 Commission/payment and pause workflow

31. System or Admin creates commission/subscription record for shop.
32. Payment status is PENDING, PAID, OVERDUE or WAIVED.
33. If unpaid beyond allowed period, Admin can pause shop or system can recommend pause.
34. Paused shop may be hidden or marked unavailable based on platform rule.
35. After payment, Admin reactivates shop and audit log is created.

Backend mapping

Required backend: commission records, payment status, shop pause reason, status transition logs.

6.8 Review moderation workflow

36. Customer submits review/rating for shop or product.
37. System stores review as PUBLISHED or PENDING_MODERATION depending on setting.
38. Admin can hide, approve or reject flagged reviews.
39. Shop Owner may view reviews and optionally reply later.
40. Public rating aggregates update after approved reviews.

Backend mapping

Required backend: review model, moderation status, aggregate rating calculation.

7. Screen and Backend Mapping

Role	Screen	Purpose	Primary backend modules
Admin	Login	Secure admin access.	auth, users
Admin	Dashboard	Platform snapshot and pending actions.	shops, support, analytics, commissions
Admin	Shops	Create/manage shops and statuses.	shops, shop-owners, locations, audit-logs
Admin	Shop Detail	Review all shop data and owner assignment.	shops, products, hours, media, reviews
Admin	Categories	Manage platform categories/tags.	categories
Admin	Users/Owners	Manage accounts and role assignments.	users, roles, shop-owners
Admin	Support	Handle support tickets.	support-tickets, notifications
Admin	Settings	Platform rules and feature flags.	settings
Owner	Owner Dashboard	Shop status, setup checklist and quick actions.	shops, products, analytics, support
Owner	Shop Profile	Edit public business information.	shops, locations, media
Owner	Business Hours	Manage schedule and closures.	business-hours
Owner	Products	Manage products/services.	products, categories, media
Owner	Support	Raise and track tickets.	support-tickets
Customer	Home	Browse local categories and shops.	discovery, categories, shops
Customer	Search	Find shops/products.	search, shops, products
Customer	Shop Detail	View shop info, products, reviews and location.	shops, products, reviews, favorites
Customer	Product Detail	View item details and shop association.	products, media, favorites
Customer	Favorites/Profile	Personal saved items and settings.	users, favorites

8. Backend-First Roadmap

Roadmap rule

Build backend in phases. Do not ask Codex to build all modules in one task. One Codex task should target one phase or one module.

Phase	Name	Goal	Key modules	Exit criteria
0	Foundation	Create Nx/NestJS backend skeleton and shared packages.	api, config, db, common	App boots, health check works, env validation ready.
1	Auth and RBAC	Secure identity, sessions and role guards.	auth, users, roles	Admin, owner and customer auth flows defined and tested.
2	Admin shop core	Admin can create/manage shops and owners.	shops, shop-owners, invites, audit-logs	Admin can create shop, assign owner and change status.
3	Shop owner setup	Owner can complete shop profile.	locations, business-hours, media	Owner can update own shop profile, location and hours.
4	Catalogue	Owner can manage categories/products/media .	categories, products, product-media	Products can be created, listed, updated and published.
5	Discovery	Customers can browse/search active shops/products.	search, favorites, reviews	Public APIs return active listings with filters.
6	Operations	Support, notifications, audit and analytics.	support, notifications, analytics	Tickets and notifications work for admin/owner/customer.
7	Commercial	Commission/payment and pause logic.	commissions, payments, subscriptions	Overdue/pause/ reactivation workflow implemented.

8.1 MVP Recommendation

- MVP backend should include Phases 0 to 5 completely.
- Support tickets from Phase 6 are strongly recommended because shop owners need help flow early.
- Commission tracking can start as admin-managed records before integrating payment gateway.
- Reviews can be basic in MVP or delayed if product launch needs speed.
- Notifications can start as database records before push/email delivery.

8.2 Build order for first backend sprint

41. Set up Nx workspace and NestJS API app.
42. Add Prisma/PostgreSQL package and base schema from DB Blueprint.
43. Create common API response, error handling, validation pipe and config module.
44. Implement Auth, Users and Roles.
45. Implement Admin Shop Management: create/list/detail/update/status.
46. Implement Shop Owner relation and invitation/assignment.
47. Implement Location and Business Hours.
48. Implement Category and Product modules.
49. Implement public discovery read APIs.
50. Add support ticket module and audit logs.

9. Product-to-Backend Module Mapping

Product capability	Backend modules	Important models	Notes for Codex
Authentication	auth, users, roles	User, Role, RefreshToken, Invite	Implement first. Every module depends on current user and role.
Admin shop management	shops, shop-owners	Shop, ShopOwner, ShopStatusHistory	Admin can access all shops; owner only assigned shops.
Location and discovery	locations, search	ShopLocation, UserLocationPreference	Add indexes for city/area/coordinates/search fields.
Business hours	business-hours	BusinessHour, SpecialBusinessHour	Compute open/closed status in service helper.
Catalogue	categories, products, media	Category, Product, ProductMedia, ShopMedia	Support tags array and category relations.
Customer actions	favorites, reviews	FavoriteShop, FavoriteProduct, Review	Require customer auth except public reads.
Support	support-tickets	SupportTicket, SupportTicketMessage	Tickets can belong to user and optionally shop.
Commercial	commissions, payments, subscriptions	Commission, PaymentTransaction, SubscriptionPlan	Can start admin-managed; payment gateway later.
Operations	notifications, audit-logs, settings	Notification, AuditLog, PlatformSetting	Useful for reliability and future automation.

9.1 API Style Rules

- REST API first. Avoid GraphQL for MVP.
- Admin endpoints should use /admin prefix where useful.
- Owner endpoints should be scoped by authenticated owner and assigned shop.
- Public discovery endpoints should only return ACTIVE shops/products.
- All write endpoints must validate DTOs and check permissions.
- All important state changes should create audit logs.

9.2 Example endpoint grouping

```

Auth
POST  /auth/login
POST  /auth/register-customer
POST  /auth/refresh
POST  /auth/logout
GET    /auth/me

Admin Shops
GET    /admin/shops
POST   /admin/shops
GET     /admin/shops/:id
PATCH /admin/shops/:id
PATCH /admin/shops/:id/status
POST   /admin/shops/:id/owners

Owner Shop
GET     /owner/shops
GET     /owner/shops/:shopId
PATCH  /owner/shops/:shopId/profile
PUT     /owner/shops/:shopId/location
PUT     /owner/shops/:shopId/business-hours

Products
GET     /owner/shops/:shopId/products
POST    /owner/shops/:shopId/products
PATCH  /owner/shops/:shopId/products/:productId
DELETE  /owner/shops/:shopId/products/:productId
GET     /public/products
GET     /public/products/:slug

Discovery
GET     /public/categories
GET     /public/shops
GET     /public/shops/:slug
GET     /public/search

```

10. Data Ownership and Permission Rules

Rule	Meaning	Implementation
Admin global access	Admin can view/manage platform data based on permission.	Admin role guard plus service-level checks.
Owner scoped access	Shop Owner can only manage shops assigned to them.	Check ShopOwner relation before every owner write.
Customer personal access	Customer can only manage own profile, favorites, reviews and tickets.	Use current user id from JWT.
Public read safety	Guest/customer can only read ACTIVE/PUBLISHED public data.	Service filters must enforce status.
No blind deletes	Prefer soft delete/archive for shops/products/categories.	Use archivedAt/status fields where possible.
Audit important writes	Status changes, owner assignment, commission edits, moderation should be logged.	AuditLogService called from services.
Search data normalization	Slugs, keywords, tags, category ids and location fields must be normalized.	Create helper functions and indexes.

10.1 State Transition Rules

Entity	Allowed transition examples	Not allowed without admin
Shop	DRAFT -> PENDING_REVIEW -> ACTIVE; ACTIVE -> PAUSED; PAUSED -> ACTIVE; ACTIVE -> SUSPENDED	Owner cannot self-activate a suspended/paused shop if admin paused it.
Product	DRAFT -> ACTIVE; ACTIVE -> INACTIVE; ACTIVE -> OUT_OF_STOCK; any -> ARCHIVED	Owner cannot publish product if shop is not active depending on rule.
Ticket	OPEN -> IN_PROGRESS -> RESOLVED -> CLOSED; OPEN -> WAITING_FOR_USER	Requester should not force-close admin-side workflow unless allowed.
Commission	PENDING -> PAID; PENDING -> OVERDUE; OVERDUE -> PAID/WAIVED	Owner cannot mark own commission as paid.

11. Codex Usage Plan

Yes, feed this first

This Product/Roles/Workflow PDF should be the first context file Codex reads. Then feed the DB Blueprint and NestJS Backend Blueprint. Product context comes first so Codex understands WHY modules exist before it edits code.

11.1 Recommended context order

51. localo_product_roles_workflows_roadmap.pdf - product meaning, roles, workflows and roadmap.
52. localo_complete_database_blueprint.pdf - database tables, columns, relations and indexes.
53. localo_nestjs_backend_blueprint.pdf - NestJS code standards, modules, APIs, DTOs and implementation rules.
54. Current repository files - only after the task is clearly defined.

11.2 Codex task rules to reduce usage

- Do not ask Codex to brainstorm product decisions; those decisions live in this PDF.
- Do not ask Codex to scan the entire repo unless the task requires it.
- One Codex task should implement one phase or one module only.
- Tell Codex exactly which files/modules it may edit.
- Ask Codex to stop after implementation and list changed files.
- Ask Codex to run only relevant typecheck/tests, not unnecessary full-suite commands every time.

- Keep UI out of backend tasks unless the task explicitly needs UI integration.

11.3 Codex prompt templates

Prompt 1 - Backend foundation

Read the three Localo PDFs in this order: Product/Roles/Workflow, DB Blueprint, NestJS Backend Blueprint.
Implement only Phase 0: Nx/NestJS backend foundation.
Create or update only backend/config/db/common files needed for foundation.
Do not implement business modules yet.
Run the minimal build/typecheck needed and list changed files.

Prompt 2 - Auth and RBAC

Use the Localo PDFs as source of truth.
Implement only Phase 1: Auth, Users, Roles, JWT refresh flow and role guards.
Do not implement shops/products yet.
Include DTOs, service, controller, guards, Prisma models/migrations if missing.
Run relevant tests/typecheck only.

Prompt 3 - Admin shop core

Use the Localo PDFs as source of truth.
Implement only Phase 2: Admin Shop Management and Shop Owner assignment/invites.
Admin can create, list, read, update and change shop status.
Owner can only be assigned, not self-register shop in MVP.
Do not build UI. Stop after backend implementation and list changed files.

Prompt 4 - Catalogue

Use the Localo PDFs as source of truth.
Implement only Categories, Products, Product Media and owner-scoped product CRUD.
Ensure public APIs only return ACTIVE/PUBLISHED data.
Do not modify unrelated modules.

12. Product Roadmap

Release	Theme	Included features	Success signal
R0	Internal backend foundation	Auth, users, roles, DB schema, migrations, health check, Swagger.	Backend deploys and core auth works.
R1	Admin-managed shops	Admin creates shops, owners, locations, statuses, categories.	Admin can onboard first real shops.
R2	Shop owner portal backend	Owner profile, hours, products, media and support tickets.	Owners can maintain their shop data without admin.
R3	Customer discovery backend	Public search, shop detail, product detail, favorites, reviews.	Customer app can browse real listings.
R4	Operations	Support queue, notifications, audit logs, basic analytics.	Admins can operate platform daily.
R5	Commercial control	Commission, payment status, overdue/pause/reactivate logic.	Platform can manage revenue and unpaid shops.
R6	Growth	Featured shops, promotions, advanced analytics, campaigns, SEO pages.	Localo can support marketing and expansion.

12.1 Frontend after backend

- Build UI after backend contracts are stable.
- Start with Admin desktop pages because Admin creates shops in MVP.
- Then build Shop Owner screens for profile, hours and products.
- Then build Customer mobile discovery screens.
- Use Swagger/OpenAPI from backend to guide frontend integration.

12.2 Decision backlog

Decision	Default recommendation	When to revisit
Can shop owners self-register?	No for MVP. Admin creates shops first.	After operations and verification are mature.
Are products physical goods, services or both?	Support generic product/service listing fields.	When vertical-specific flows are needed.
Do we support checkout?	No in MVP.	After discovery and shop onboarding work well.
Do we require reviews in MVP?	Optional. Basic rating/review can be Phase 5.	If launch needs faster scope, defer.
Do we auto-pause unpaid shops?	Start with admin-controlled pause.	Automate after payment workflow is stable.
Do we need Redis/search engine early?	No for MVP unless search becomes slow.	After data size grows.

13. Backend Acceptance Criteria

- All APIs are protected by correct public/authenticated/admin/owner/customer access rules.
- Owner-scoped APIs cannot access another owner shop.
- Public APIs never expose inactive, suspended or archived shops/products.
- DTO validation prevents bad price, discount, email, phone, location and status data.
- Soft delete/archive is used for major business entities where historical data matters.
- Important actions create audit log entries.
- Swagger/OpenAPI documentation is generated for controllers and DTOs.
- Migrations are deterministic and seed data is available for local development.
- Error format is consistent across modules.
- Pagination, filtering and sorting are consistent on list endpoints.

13.1 MVP demo checklist

Demo	Expected result
Admin logs in	Admin receives token and can call admin endpoints.
Admin creates shop	Shop is saved as DRAFT/PENDING and visible in admin list.
Admin assigns owner	Owner can log in and see only assigned shop.
Owner updates profile	Shop public details update but remain status-controlled.
Owner creates product	Product appears in owner product list and public list only when active.
Customer searches	Search returns active shops/products with category/location filters.

Customer saves favorite	Favorite is saved under current customer only.
Owner raises ticket	Ticket appears in admin support queue.
Admin pauses shop	Paused shop no longer behaves as active in public APIs.

14. Glossary

Term	Meaning
Localo	The local shop discovery and management platform.
Shop	A business listing managed by Admin and/or Shop Owner.
Shop Owner	User who manages one or more assigned shops.
Customer/User	End user who browses and interacts with public listings.
Guest	Unauthenticated visitor with limited access.
Category	Grouping used for browsing, filtering and search.
Tag	Flexible keyword used to improve search and discovery.
Business Hours	Weekly and special schedule used to compute open status.
Commission	Commercial charge/fee or revenue tracking record for a shop.
Paused Shop	Shop temporarily not active due to admin decision, overdue payment or other reason.
Audit Log	Record of important actions for traceability.

15. Final Notes for Implementation

- Use this PDF as product truth, not as database truth. The DB Blueprint remains database truth.
- Use the NestJS Backend Blueprint as code structure/API/DTO truth.
- When a conflict appears, prefer product intent from this PDF, data details from DB PDF, and implementation style from Backend PDF.
- If Codex is uncertain, it should stop and ask instead of inventing new modules or business flows.
- Backend comes first. UI should consume stable API contracts after core backend is ready.